

DIAMesh 操作手冊

Delta Intelligence Agent for **Mesh** processing — Python 3D FBX viewer + 自動 mesh reduction toolkit · 建在 [pyrender](#) 之上。

本手冊涵蓋：

- **問題背景** — DIAMesh 解什麼問題、為什麼需要它
- **架構** — pyrender 內嵌 + 三組 reducer backend + 跨平台 vendor binary
- **CLI 指令** — `view` / `info` / `reduce` 完整參考
- **Backend 選擇** — trimesh / pymeshlab / blender 三條路線的 trade-off
- **Mesh Repair Pipeline** — 從 weld 到 island cull 的七階段內部流程
- **Production LOD Presets** — 30 台設備產線 viewer 的三檔推薦配置 (廠區/單機/Hero)
- **跨平台部署** — Windows 即用、Linux/macOS 一行 setup
- **故障排除** — 常見錯誤跟修法

如要快速看路線圖請翻 [ROADMAP.md](#) 。

目錄

0. 問題背景與方案概覽
 1. Pipeline 全貌
 2. 環境設定
 3. CLI 指令參考
 4. Backend 選擇指南
 5. Mesh Repair Pipeline 內部解剖
 6. Production LOD Presets
 7. 跨平台部署
 8. 故障排除
 9. 限制與未來工作
-

0. 問題背景與方案概覽

0.1 要解決的問題

工業 3D viewer 的核心痛點：一條產線 30 台設備同畫面渲染就卡。每台設備是 100k+ face 的 CAD-export FBX，30 台 = 3M+ face → 即時渲染崩潰。

但需求又不能讓細節全砍：單台設備 zoom-in 看仍要保留可辨識的結構——LOD (Level of Detail) 系統的經典場景。

DIAMesh 處理的就是這個跨度：* FBX 載入 + 預覽 (Phase 1) — 看清楚原始 mesh * 自動 mesh reduction (Phase 2) — 砍 face 但保結構連續性 * LOD-friendly 輸出 (Phase 2.5) — 拋掉真正看不到的微結構，主結構完整

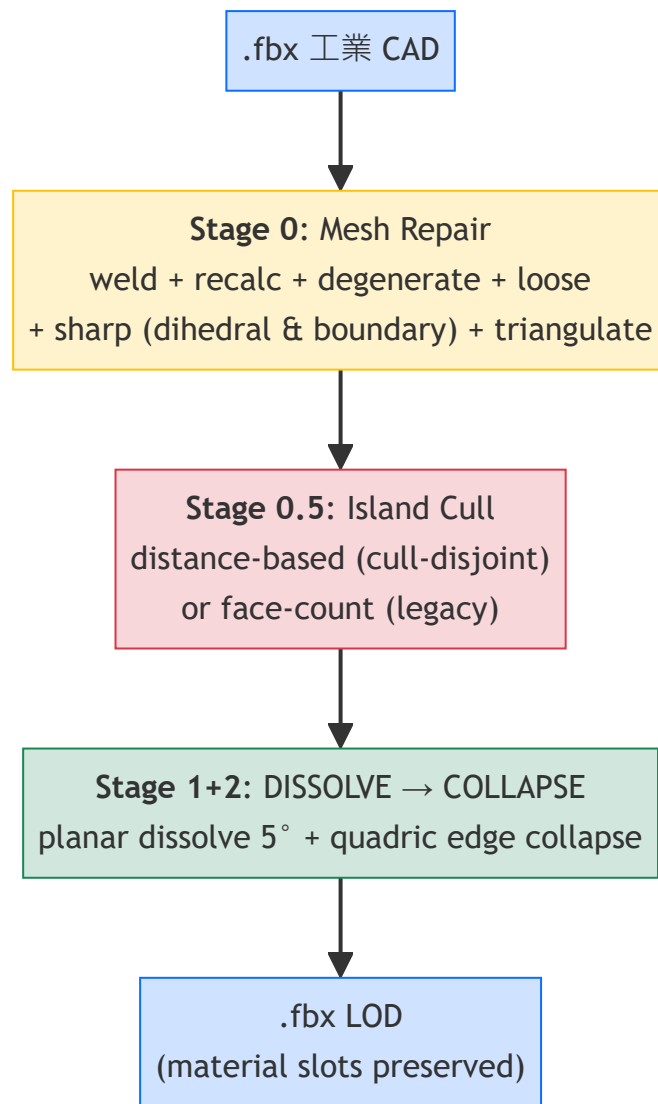
0.2 為什麼選 pyrender 為基底

幾個 Python 3D viewer 候選：

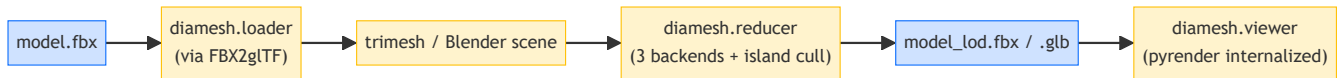
工具	評估
pyrender	✔ 純 Python、MIT、PBR rendering、scene graph 可擴展、active project
vedo	VTK-based，重，互動體驗一般
open3d	強大但偏 point cloud / vision 應用，FBX 支援不直接
trimesh.Scene().show()	太簡單，作 viewer 不夠正式
panda3d	遊戲引擎重型

DIAMesh 選 C 路 (內嵌複製 pyrender 源碼) —— 把 pyrender 當 DIAMesh 自家代碼，未來客製化 (heatmap shader、整合 GUI) 可直接改 internals。

0.3 三段式 mesh reduction 流程



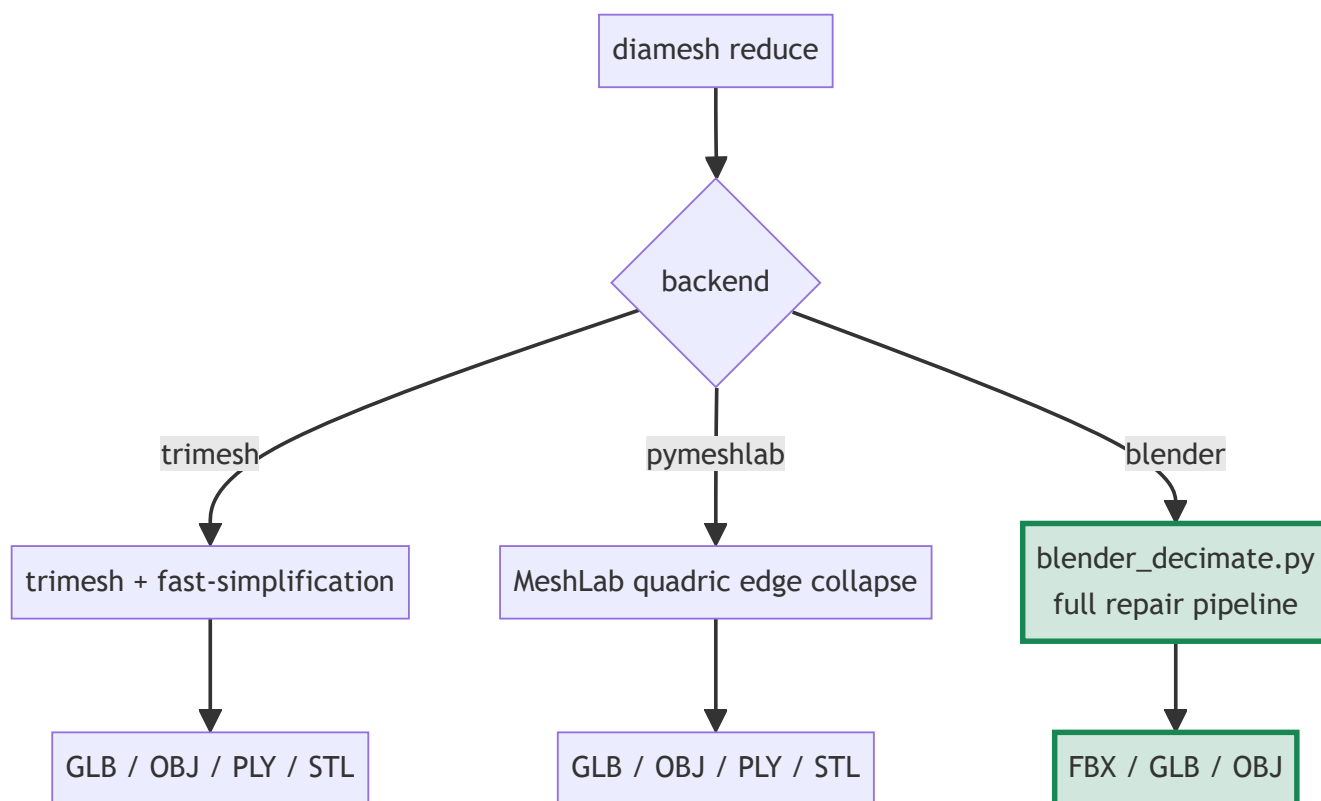
1. Pipeline 全貌



1.1 倉儲結構

```
DIAMesh/
├─ pyrender/          # ★ 內嵌的 mmatl/pyrender 源碼 (MIT 授權保留)
├─ diamesh/           # ★ DIAMesh 自家層
│  ├─ loader.py       # FBX → trimesh 透過 vendored FBX2glTF
│  ├─ viewer.py       # 包 pyrender.Viewer
│  ├─ reducer.py      # mesh reduction 三 backend dispatcher
│  └─ cli.py          # `diamesh view|info|reduce`
├─ scripts/
│  ├─ blender_decimate.py # Blender headless 全套 pipeline
│  └─ setup_vendor.py    # Linux/macOS 一鍵下載 binary
├─ vendor/
│  ├─ fbx2gltf/         # FBX2glTF v0.9.7 (MIT)
│  ├─ assimp/           # Assimp v6.0.5 (BSD-3)
│  ├─ blender/          # Blender Portable (gitignored, manual)
│  ├─ PYRENDER_LICENSE.md
│  ├─ FBX2GLTF_LICENSE.md
│  ├─ ASSIMP_LICENSE.md
│  └─ BLENDER_SETUP.md
└─ tests/fixtures/
    └─ sphere.fbx      # 自動生成的 unit-test fixture
```

1.2 兩條 reducer 主路線



Blender backend 是 production 推薦 —— 唯一保留材質/材質貼圖/層級結構、唯一支援 island cull 跟全套 mesh repair。trimesh / pymeshlab 是輕量替代給快速實驗用。

2. 環境設定

2.1 安裝

內網用戶取得專案的方式：DIAMesh 不對外公開，內網其他用戶請向開發團隊（James 老大 / 小福）索取 DIAMesh 最新 zip 檔。將 zip 解壓到本機任一目錄即可。

```
cd <unpacked-DIAMesh-directory>
pip install -e .
```

2.2 平台特定 vendor binary

Windows 用戶：FBX2glTF.exe 跟 Assimp DLL 已經 git-tracked，clone 即用。

Linux / macOS 用戶：跑一次 setup script 自動下載對應 binary：

```
python scripts/setup_vendor.py
```

該腳本會：1. 偵測 `platform.system()` 跟 `platform.machine()` 2. 下載 `FBX2glTF v0.9.7` 對應 release 到 `vendor/fbx2gltf/` 3. 下載 `Assimp v6.0.5` 對應 release 並解出 shared library 到 `vendor/assimp/` 4. 自動 `chmod 0o755`（POSIX 系統的執行權限）5. 已存在則跳過（idempotent）

2.3 Blender 安裝（用 `backend=blender` 才需要）

Blender Portable 太大（約 250 MB）+ GPL 授權考量，DIAMesh 不自動下載 Blender。手動部署：

1. <https://www.blender.org/download/> 下載 portable / zip / tar.xz / dmg（建議 LTS 4.2.x）
2. 解壓 / 拷貝到 `vendor/blender/` 即可（DIAMesh 自動偵測 `vendor/blender/blender.exe` 或 `vendor/blender/blender`）
3. 或設環境變數 `BLENDER_EXE=/path/to/blender`

詳細指引見 [vendor/BLENDER_SETUP.md](#)。

2.4 驗證安裝

```
diamesh --help          # CLI 主入口
diamesh info tests/fixtures/sphere.fbx
# file: tests/fixtures/sphere.fbx
#   n_meshes: 1
#   total_vertices: 642
#   total_faces: 1280
#   ...
```

3. CLI 指令參考

3.1 `diamesh view <file>`

開啟互動式 3D viewer 顯示 mesh。

```
diamesh view data/Robot.fbx
```

鍵盤操作： | key | 行為 | |---|---| | 左鍵拖 | 旋轉 | | 中鍵拖 / Shift+左鍵拖 | 平移 | | 滾輪 / 右鍵拖 | 縮放 | | P | 存截圖 (注意：原 pyrender 用 S，DIAMesh 改 P 避免跟 Win+Shift+S 衝突) | | R | 開始/停止 GIF 錄影 | | W | 切 wireframe 模式 | | H | 切 shadows | | F | 切 fullscreen | | Z | reset 視角 | | Q | 離開 |

3.2 `diamesh info <file>`

不開窗，印 mesh 統計資料。

```
diamesh info data/Robot.fbx
```

輸出：mesh 數、頂點總數、面數總數、watertight 計數、各 mesh 邊界框。

3.3 `diamesh reduce <file>`

自動 mesh reduction。

基本用法：

```
diamesh reduce data/Robot.fbx --target-faces 5000
diamesh reduce data/Robot.fbx --ratio 0.25 -o data/Robot_lod.glb
```

完整參數： | flag | type | 說明 | |---|---|---| | `--target-faces N` | int | 目標 face 數量 (與 `--ratio` 二選一) | | `--ratio R` | float | 保留 face 比例 0..1 (與 `--target-faces` 二選一) | | `--output / -o PATH` | str | 輸出檔 (預設 `<input>_reduced.glb`) ; 副檔名決定格式 | | `--backend {trimesh,pymeshlab,blender}` | str | reducer backend (預設 trimesh) | | `--cull-disjoint THRESHOLD` | float | (blender) 距離主結構 > 此比例的 island 刪除 (預設 0 = disabled) | | `--cull-anchor-count N` | int | (blender) 取最大 N 個 island 為 anchor (預設 10) | | `--min-island-faces N` | int | (blender, 粗暴) 砍 face < N 的島 (預設 0) |

輸出格式： * `.fbx` — 完整保留 material (要 `--backend blender`) * `.glb` — 預設，PBR material 完整保留 * `.obj` / `.ply` / `.stl` — 純幾何，丟材質

4. Backend 選擇指南

4.1 三 backend 比較

維度	trimesh	pymeshlab	blender ★
速度	⚡ 快	慢	慢
安裝	內建	<code>pip install pymeshlab</code>	手動下 Blender Portable
Material 保留	✗ 全丟	✗ 全丟	✓ 完整
Texture 保留	✗	✗	✓ embed
部件層級保留	✗	✗	✓ → 1 mesh + multi-material
Boundary 保護	✗	✓	✓
Sharp edge 保護	✗	✗	✓
Mesh repair	✗	✗	✓ 完整 7 階段
Island cull	✗	✗	✓
FBX output	✓ via assimp	✗	✓
適用場景	smoke test、快速實驗	高品質但無 material 需求	生產 LOD

4.2 推薦選擇

- 要 material / production LOD → `--backend blender`
- 快速 smoke test 不在乎材質 → `--backend trimesh` (預設)
- 要 boundary 保護但不要 Blender → `--backend pymeshlab`

5. Mesh Repair Pipeline 內部解剖

只在 `--backend blender` 啟用，由 `scripts/blender_decimate.py` 在 Blender headless 內執行。

5.1 全流程



Syntax error in text
mermaid version 10.9.5

5.2 為什麼 JOIN ALL 是關鍵

工業 CAD-export FBX 常見「同位置、不同 mesh object、不同 vertex index」的接觸面 vertex —— 視覺上貼合，topology 上是兩條獨立 edge。reduce 時兩個 vertex 各自 collapse 不同方向 → 接觸面分離 → 漂浮碎片。

`bpy.ops.object.join()` 把所有 mesh objects 合併成單一 mesh，per-face material_index 自動保留，配合 Stage 0 的全局 weld 一次解決 cross-part 對齊。

5.3 Stage 0 各步驟的功能

步驟	bmesh op	解決什麼
A. Recalc normals	<code>recalc_face_normals</code>	CAD-export 法向量翻轉 → 讓 NORMAL delimit 正確
0. Weld (adaptive)	<code>remove_doubles(dist=bbox_diag × frac)</code>	CAD-patch seam 跟 cross-part 接觸面對齊。 Tolerance 自動縮放：2 m 機台 ≈ 0.1 mm、5 m 機台 ≈ 0.25 mm、30 cm 部件 ≈ 0.015 mm。可用 <code>--weld-tolerance-frac</code> (default 5e-5) 或 <code>--weld-tolerance-abs</code> 覆寫
B. Dissolve degenerate	<code>dissolve_degenerate(dist=1e-5)</code>	零面積 sliver 三角形
C. Loose cull	<code>delete(geom=loose)</code>	孤立 vertex/edge → 直接消除「漂浮碎片」一部分來源
D. Mark sharp + boundary	<code>edge.smooth = False</code>	(1) 30° 以上 dihedral edge mark sharp → SHARP delimit 才生效；(2) 每條 boundary edge (1-face edge) 也 mark sharp → COLLAPSE 不再破壞 boundary，從源頭減少 decimation 製造的破洞（治本）
E. Triangulate	<code>triangulate(BEAUTY/BEAUTY)</code>	混合 quad/ngon → 純 tri，COLLAPSE 行為一致
F. Non-manifold fix (opt-in)	<code>dissolve_edges(non-manifold)</code>	啟用 <code>--fix-non-manifold</code> 時，dissolve 共享 ≥3 face 的邊。預設關閉 — 部分 CAD 設計依賴 3-patch junction，誤刪會破壞拓樸；當 <code>diamesh diagnose</code> 報 <code>non_manifold_edges > 0</code> 時值得試

5.4 Stage 0.5 — Distance-based Island Cull

CAD assemblies 常含「結構上 disjoint 但視覺上貼合」的 sub-component（螺絲蓋、感測器探頭等）。原 mesh 因為 dense triangulation 視覺包圍它們所以看起來貼合，reduce 後砍掉周圍 → ground truth 露出 → 漂浮碎片。

`--cull-disjoint THRESHOLD` 演算法：1. BFS 找 connected face islands 2. 取最大 N 個 island（預設 N=10）為 anchor 3. 對其他 island 計算 bbox 到任一 anchor 的最近距離 4. 距離 / 整體 mesh 對角線 > threshold → 真漂浮 → 刪 5. 接觸 anchor 的 → 保留（即使 face 少）

關鍵點：主框架的金屬桿往往也是 disjoint island（每根 50-200 face），但 bbox 接觸面板 → 被保留。螺絲蓋 bbox 離主結構幾 mm → 刪。

threshold 調參指南：* `0.01` — 只刪離得很遠的（保留接觸或極近的零件）* `0.02` — 推薦起點（細節保留 vs 漂浮清除的平衡）* `0.03` — 中等 * `0.05+` — 寬鬆

5.5 Stage 1+2 — Two-pass Decimation

Stage 1 DISSOLVE (planar/limited dissolve): * `angle_limit = 5°` : 5° 以內的相鄰面合併為 n-gon *
`delimit = {NORMAL}` : 不跨法向量斷層合併 * **視覺無損** : 純消除冗餘三角化, silhouette 不動

Stage 2 COLLAPSE (quadric edge collapse): * `ratio = target_faces / post-dissolve faces` *
`delimit = {MATERIAL, SHARP, SEAM}` : 不跨材質邊、銳邊、UV seam * `use_collapse_triangulate = True` : 結果保 tri

兩階段串聯: DISSOLVE 先消化平面冗餘 (lossless) , COLLAPSE 才動 silhouette geometry → 對工業 CAD 板狀結構特別有效。

6. Production LOD Presets

TL;DR — 一行指令搞定：

```
diamesh reduce machine.fbx --preset tier1 -o out.fbx    # 廠區/30 機
diamesh reduce machine.fbx --preset tier2 -o out.fbx    # 單機聚焦
diamesh reduce machine.fbx --preset tier3 -o out.fbx    # Hero shot
diamesh reduce machine.fbx --preset balanced -o out.fbx # = tier2 alias
```

每個 preset 內部展開成「主 backend + ratio + cull-disjoint + smart fill + bridge loops + 內建 boundary preservation」的完整 production pipeline。詳見 §6.1/6.2/6.3 各檔說明。

6.0 設計原則 — 完整 > 邊緣連續 > 不破面 > 細節

LOD 場景下，**主體完整性** 永遠比 **細節豐富度** 重要：

- zoom out 看 30 台時，洞和漂浮會變成「視覺雜訊」干擾整廠認知
- 一台斷面的設備比一台簡化的設備更糟糕
- 細節可以靠 LOD 切換補回（近距離換 TIER 2/3），但破面回不了

三檔 preset 都遵守這個原則：先保完整，再談 face budget。

6.1 TIER 1 — 廠區/產線視角（30 台同畫面）

```
diamesh reduce data/Machine.fbx --preset tier1 -o data/Machine_TIER1.fbx
```

展開為：--backend blender --ratio 0.1 --cull-disjoint 0.025 --auto-fill-holes --fill-holes-skip-design --bridge-loops

特性：* face 砍到 ~10% (input 75k → output ~7k) * 30 台 × 7k = ~210k face / 整條產線 * 主框架完整、漂浮乾淨清空、面板實心 * GPU draw call 友善 (單 mesh + multi-material)，筆電內顯也能跑

適用視角：* 整廠 / 整條產線 zoom-out * 上線監控大屏 (多視窗多產線同時顯示) * CEO Dashboard 全廠狀態圖 * AR/VR 漫遊時的中遠景

用戶體驗目標：使用者一眼能數出有幾台設備、看得出設備種類分佈、不會被破面噪訊干擾。

6.2 TIER 2 — 單機聚焦（點選展開細節）

```
diamesh reduce data/Machine.fbx --preset tier2 -o data/Machine_TIER2.fbx
# or alias:
diamesh reduce data/Machine.fbx --preset balanced -o data/Machine_TIER2.fbx
```

展開為：`--backend blender --ratio 0.25 --cull-disjoint 0.04 --auto-fill-holes --fill-holes-skip-design --bridge-loops`

特性：

- * face 砍到 ~25% (input 75k → output ~17k)
- * 30 台 × 17k = ~510k face / 整條產線 (如全切換為 TIER 2)
- * 機械手臂、HMI 螢幕、子結構可辨識
- * 中等 GPU (GTX 1660 等級) 仍流暢

適用視角：

- * 用戶在 viewer 點擊「這台」展開查看
- * 故障設備聚焦 (MES / SCADA 整合彈出)
- * 維修人員 AR 標註

用戶體驗目標：保留足夠細節讓使用者能從外觀辨識設備型號、看清 HMI 與機械臂的相對位置。

6.3 TIER 3 — Hero Shot (行銷素材 / CEO 簡報)

```
diamesh reduce data/Machine.fbx --preset tier3 -o data/Machine_TIER3.fbx
```

展開為：`--backend blender --ratio 0.5 --cull-disjoint 0.03 --auto-fill-holes --fill-holes-skip-design --bridge-loops`

特性：

- * face 砍到 ~50% (input 75k → output ~37k)
- * 視覺接近原檔，材質、銘牌、HMI 螢幕細節都保留
- * 適合單台設備獨立渲染

適用視角：

- * 行銷素材 / 產品冊 / 官網設備介紹
- * CEO 簡報、客戶 demo、法說會
- * 數位雙生展示影片的特寫鏡頭
- * 印刷高解析度單機產品圖

用戶體驗目標：「看起來幾乎沒減面」— 但檔案小一半，傳輸/載入快一倍。

6.4 三檔 LOD 切換策略

production viewer 一次生成三份：

```
# Windows
for %F in (data\Machine.fbx) do (
    diamesh reduce %F --preset tier1 -o %~dpnF_TIER1.fbx
    diamesh reduce %F --preset tier2 -o %~dpnF_TIER2.fbx
    diamesh reduce %F --preset tier3 -o %~dpnF_TIER3.fbx
)

# Linux/macOS
for f in data/*.fbx; do
    diamesh reduce "$f" --preset tier1 -o "${f%.fbx}_TIER1.fbx"
    diamesh reduce "$f" --preset tier2 -o "${f%.fbx}_TIER2.fbx"
    diamesh reduce "$f" --preset tier3 -o "${f%.fbx}_TIER3.fbx"
done
```

viewer 根據 camera 距離自動切換 LOD：

Camera 距離	載入哪檔	視覺體驗
遠 (產線俯瞰)	TIER 1	主體輪廓完整
中 (單機聚焦)	TIER 2	結構可辨識
近 (特寫鏡頭)	TIER 3 / 原檔	細節豐富

這是業界 LOD 系統的標準應用方式 (Unreal / Unity 都這樣做) 。

6.5 為什麼這三組是甜蜜點 — 9 組對比實測

針對 5AxisGlueSpraying.fbx (75k face, 24 part) 的完整對比，三軸：ratio × cull-disjoint × fill-holes-max-sides。

編號	ratio	cull	fill	主體完整	邊緣連續	漂浮	細節	判定
A1	0.1	0.02	8	✓	✓	△ 1個	中	OK
A2	0.1	0.025	8	✓	✓	✓ 乾淨	中	🏆 TIER 1
A3	0.1	0.02	4	△ 透空	△	△	中	fill 太保守
A4	0.1	0.02	16	✓	✓	△	中	跟 fill8 沒差
B1	0.25	0.03	8	✓	✓	△	高	OK
B2	0.25	0.025	8	✓	✓	△ 多	高	cull 太低
B3	0.25	0.04	8	✓	✓	✓ 乾淨	高	🏆 TIER 2
X1	0.05	0.02	8	△ 透	✓	△	低	太極限
X2	0.5	0.03	8	✓	✓	△	最高	🏆 TIER 3

三個關鍵 insight：1. fill-holes-max-sides=8 是甜蜜點 — 4 太保守 (面板透空)、16 跟 8 差異很小 (沒必要)。詳見 §6.6 2. cull threshold 隨 ratio 縮放 — ratio 越高 (保留越多面)，cull 可以更激進 (c0.04)，因為主體 anchor 結構還在 3. 沒有單一最佳 — 三檔各司其職，這正是 production LOD 系統需要的分層

6.6 為什麼 --fill-holes-max-sides 8

decimation 過程的 quadric edge collapse 會在 boundary 邊緣 (CAD seam、不同 part 交界) 產生小的 boundary loop (沒有 face 蓋上的開放邊緣)，視覺上呈現「面板透空」或「機殼缺角」。

--auto-fill-holes 在 decimation 後對所有 boundary loop 執行 Blender 的 bpy.ops.mesh.fill_holes(sides=N) — 把長度 ≤ N 的 loop 用三角形補上。

N	適用	風險	結果
4	只補小四邊洞	大洞補不到 → 面板透空	A3 視覺較差
8	主流 CAD seam 洞、機殼小縫	機殼開口若 ≤ 8 邊也會被補	平衡點
16	大開口都補	通風孔、面板鏤空被誤封	跟 8 差異不大但風險高

何時調整： * 視覺仍有透空感、想更激進補洞 → 試 `--fill-holes-max-sides 12` * 設備有大量設計鏤空 (散熱孔、面板透視) → 降到 `--fill-holes-max-sides 4` 或 `6`，只補小洞

`--fill-holes-smooth` — 補洞處 Laplacian 平滑 (opt-in)

`fill_holes` 補上的三角形是 planar fan triangulation — 視覺上像「貼布」。`--fill-holes-smooth` 在補洞後對新增的 face + 1-ring vertex neighbours 做 Laplacian 弛緩，讓補洞處跟周圍曲率融合。

- `--fill-holes-smooth-iter N` (default 2)：迭代次數，越高越平滑、但極限會把鄰近細節也拉平
- `--fill-holes-smooth-factor F` (default 0.5)：每次 relax 強度 0–1。0.5 平衡點

何時開啟： * 機殼有大圓弧曲面、補洞處看得出「平面 bandage」→ 開啟 * 平面結構為主 (櫃體、產線框架) → 不需要、開了沒幫助也沒傷害

KPI 選用紀律 — 衡量 LOD 質量該看哪個指標

`diamesh diff` 報告四個距離指標：Hausdorff (max + 雙向)、Chamfer、Volume diff、Mean normal deviation。並非每個都適合用來比 LOD 質量：

指標	反映什麼	用來比 LOD 質量？
Hausdorff (max)	被 cull 砍掉的零件大小	✗ 不適合
Chamfer (mean)	平均面對面距離	⚠ 對「fill 補多少」敏感，不單調
Volume diff	封閉體積差異	✗ 對 non-watertight CAD 是 NaN
Mean normal deviation	平均法向角度差	✓ 真 KPI — 跟 face_count、視覺感受一致

為什麼 Hausdorff 不適合？ Hausdorff 取的是「最壞點」距離。當 reduce pipeline 用 `--cull-disjoint` 砍掉小漂浮零件，那些零件位置的最近距離會爆掉 — 三檔 LOD 的 Hausdorff 數字幾乎相同 (都 $\approx 8\%$ of diagonal)，但 LOD 質量天差地別。**Hausdorff 反映的是 cull 行為，不是 collapse 偏差。**

為什麼 Chamfer 也要小心？ Chamfer 是雙向 mean distance，當 LOD face 多到 `fill_holes` 補了原檔沒有的 geometry， $r \rightarrow 0$ 距離會拉高，**chamfer 變得不單調** (face 多反而比中間 LOD 差)。

Mean normal deviation 是唯一可信 KPI (但有例外，見下)： face 越少 → normal 越粗略 → deviation 越大。完美線性、跟 face_count 一致、跟視覺感受一致。詳見 §6.8 Case Study 的 5AxisGlueSpraying.fbx 實測 (12.06° / 6.95° / 1.61° 對應 TIER 1 / 2 / 3)。

實用建議： * 看 LOD 質量 → 用 `mean_normal_dev_deg` * 看「reduce 砍掉了什麼」→ 用 Hausdorff (但要懂它反映 cull) * 看細節保留 → 用 Chamfer (但跟 face_count 一起看)

量化 KPI 不是萬能 — 視覺驗證仍是最終 judge

反直覺的 case：bridge loops 開啟後

維度	不開 bridge	開 bridge	解讀
chamfer	0.080%	0.080%	持平
mean_normal_dev	12.06°	12.81°	⚠️ +0.75°
face_count	19970	23357	+17%
視覺輪廓完整度	中	明顯改善	✓
機械臂周圍漂浮	△	✓ 大幅減少	✓

bridge 加 face strip 連兩 loop → 新 face 的 normal 跟原檔不對齊 → normal_dev 上升。但這些新 face 補的是「視覺破口」，整體輪廓更完整。

結論：normal_dev 衡量的是「像不像原檔」，不是「視覺完整性」。對 LOD 這兩個目標可能矛盾：* 像原檔 = 細節保留高 + 可能殘破 * 視覺完整 = 補齊裂縫 + face 多一些

實用紀律：1. 快速驗證：跑 diff 看 normal_dev、chamfer 數字 2. 如果懷疑：開 viewer 視覺確認 3. 對 LOD production：視覺完整性優先於 normal_dev 數字

6.7 端到端範例：三個典型使用情境

範例 A — 30 機產線 viewer (最常見)

情境：智能工廠展廳視覺化系統，產線一條 30 台設備，使用者透過 web viewer 漫遊整廠。

```
# Step 1 - 把 30 台 CAD 檔批次轉成 TIER 1
mkdir -p data/lod1
for f in data/raw/*.fbx; do
    diamesh reduce "$f" --backend blender \
        --preset tier1 -o "data/lod1/$(basename "$f" .fbx)_lod1.fbx"
done

# Step 2 - 檢查每台 mesh 統計 ( 確認 face 預算 )
for f in data/lod1/*.fbx; do
    diamesh info "$f"
done

# Step 3 - 上 viewer / 3D engine ( Unreal / Unity / Three.js )
# 整條產線總 face budget 約 30 × 7k = 210k，筆電內顯也能流暢跑
```

預期結果：* 30 個 fbx 檔 × ~1.5 MB (含內嵌貼圖) = ~45 MB 整條產線 * 載入時間 < 5 秒 * GTX 1650 / Intel Iris Xe 跑 60 FPS 不掉

範例 B — 客戶 demo 單機聚焦

情境：客戶來訪要看某台 5 軸點膠機的細節，需要從廠區視角縮放到單機聚焦。

```
# 同一台機器產生 TIER 1 + TIER 2 兩份
diamesh reduce data/5AxisGlueSpraying.fbx --preset tier1 -o data/5Axis_TIER1.fbx
diamesh reduce data/5AxisGlueSpraying.fbx --preset tier2 -o data/5Axis_TIER2.fbx

# Viewer 根據相機距離自動切換：遠 → TIER 1，近 → TIER 2
```

預期結果：客戶 zoom-in 那台時無感切換、看到機械手臂與 HMI 螢幕都清楚。

範例 C — CEO 簡報 hero shot

情境：法說會 PPT 要放一張產線旗艦設備的高品質渲染。

```
# 用 TIER 3 — 視覺接近原檔
diamesh reduce data/5AxisGlueSpraying.fbx --preset tier3 -o data/5Axis_hero.fbx

# 在 Blender 開啟 hero.fbx → 高品質渲染 (Cycles, 4K) → PNG 導出
```

預期結果：原檔 75k face → 37k face，視覺幾乎沒差，但渲染時間少一半，PPT 可以放多張角度也不卡。

範例 D — 嚴重破爛 mesh 的搶救

情境：CAD 匯出的 FBX 一打開就一堆漂浮、重複頂點，標準 preset 救不回來。

```
# 先 info 看看狀況
diamesh info data/Bad.fbx

# 試激進 cull + 大角度 fill
diamesh reduce data/Bad.fbx \
  --backend blender --ratio 0.1 \
  --cull-disjoint 0.05 \
  --auto-fill-holes --fill-holes-max-sides 16 \
  -o data/Bad_repair.fbx
```

還救不回來時：原檔 topology 太亂，建議匯出前先在 CAD 端 export 設定改 “merged vertices” 或在 Blender 手動 weld 後再進 DIAMesh pipeline。

6.8 Case Study — 5AxisGlueSpraying.fbx 完整 baseline

這套真實 baseline 解釋了 §6.5 九組視覺實驗背後的 quantitative root cause，也驗證了 §6.0 的設計哲學在實際工業 CAD 上的表現。所有數字都來自 `diamesh diagnose` + `diamesh diff` 在 5AxisGlueSpraying.fbx 上的實際輸出，可重現。

6.8.1 原檔病理 (diagnose 報告)

```
diagnose: 5AxisGlueSpraying.fbx
face_count:      75327
vert_count:      114697      <- vert/face = 1.52
bbox_diagonal:   2.2950 m
watertight:      False
winding_consistent: True
boundary_edges:  115935      <- ≈ unique edges 總數
non_manifold_edges: 0
degenerate_faces: 52
island_count:    22552      <- 兩萬多個 island
largest_islands: [593, 589, 589, 585, 466] (top 5 = 3.7% 總面數)
inverted_normals: 0
```

三個關鍵病理 finding：

- **22,552 個 island vs 75,327 face = 平均 3.3 face/island**：大部分 island 是孤立小三角形，沒有「主結構 anchor」。top 5 islands 才 3.7% 總面數。人類視覺看是完整設備 — 僅靠 weld 0.1mm 把這些散點融合。
- **boundary_edges 115935 ≈ unique edges 總數**：幾乎每條 edge 都只屬於一個 face → 全是 boundary。這就是為什麼不加 boundary preservation，COLLAPSE 會大量破壞拓樸。
- **vert/face = 1.52 vs watertight 預期 ~0.5**：嚴重重複頂點。每個 patch seam 的頂點都被複製，0.1mm weld 是治本。

DIAMesh 的 pipeline 設計剛好對應這些病理：

病理	DIAMesh 對應 stage
22552 islands、重複頂點	adaptive weld ($5e-5 \times \text{diagonal}$ · 化萬個 island 為個位數)
仍漂浮的小 part	cull-disjoint 0.025
全是 boundary edges	boundary preservation (治本)
設計孔需保留	smart fill skip-design
52 degenerate	dissolve_degenerate
跨設備尺寸 (小部件 vs 大機台)	adaptive weld (自動 scale)
補洞處貼布感	--fill-holes-smooth (opt-in)
非流形交界	--fix-non-manifold (opt-in)

self-intersection 不在表內 — DIAMesh 只 detect (`diamesh diagnose` 報 `self_intersect_faces`) 不 fix : proper 修復要走 boolean-union remesh , 會徹底損失 UVs 跟 materials , 跟「保材質」原則撞牆。CAD 端 export 設定才是修復的對位置。

6.8.2 三檔 LOD 後的 diagnose 對比

指標	Original	TIER 1	TIER 2	TIER 3
face	75327	21305	50972	71288
ratio achieved	—	28%	67%	94%
ratio requested	—	10%	25%	50%
boundary_edges	115935	31816	75778	113938
non_manifold_edges	0	7	0	0
degenerate_faces	52	0	0	2
island_count	22552	6485	14289	23366
inverted_normals	0	8 (0.04%)	0	0
winding_consistent	True	True	False	True

Finding : face_count 達不到目標 ratio。這是 **boundary preservation + delimit 4 集合 (MAT/SHARP/SEAM/UV)** 的副作用 — 太多 edge 被保護 → COLLAPSE 砍不下去。

這是設計取捨，不是 bug：保 boundary (視覺完整 / 邊緣連續) 跟達到 face budget 兩條目標衝突。如果 face budget 是硬限制，可以放寬 delimit (例如改 `delimit={MATERIAL}` 只保護材質邊界)，但會犧牲視覺完整性。對「完整 > 細節」的 LOD 場景，當前取捨是對的。

Finding : TIER 1 偵測到 8 inverted + 7 non_manifold (0.04% / 0.02%)。極端 ratio 的 COLLAPSE 偶發 fold-over，量微小，視覺看不到，但量化指標誠實揭露。

Finding : TIER 2 winding_consistent 變 False。multi-material face 邊界處的 winding 順序不一致，不是法向方向錯 (inverted=0)，不影響視覺。

6.8.3 三檔 LOD 跟原檔的 deviation 對比

指標	TIER 1	TIER 2	TIER 3
hausdorff_max	0.187 (8.16%)	0.186 (8.12%)	0.186 (8.12%)
chamfer	0.0018 (0.080%)	0.0010 (0.045%)	0.0013 (0.055%)
mean_normal_dev_deg	12.06°	6.95°	1.61°

6.8.4 五個 quantitative insight

Insight 1 : Hausdorff 三檔幾乎一樣 (8.12%–8.16%) — 不適合用來比 LOD 質量

0.187 m / 2.295 m = 8.16% 對應的是 cull-disjoint 砍掉的小零件大小。三檔 cull 設定不同但都砍掉同類零件 (螺絲蓋、漂浮機構)，所以 Hausdorff 一致。這指標反映 cull 行為，不是 collapse 偏差。

Insight 2 : Chamfer 不單調 (TIER 2 < TIER 3 < TIER 1)

違背「face 越多越像」的直覺。解釋：TIER 3 雖 face 多但 fill_holes 補的位置貢獻 r→o 距離；TIER 2 取得 fill 跟保形的最佳平衡。這個現象只有量化指標看得出來，視覺判斷不到。

Insight 3 : Mean normal deviation 是 LOD 質量的真 KPI

12.06° / 6.95° / 1.61° 完美單調、跟 face_count 一致、跟視覺感受一致。是三個指標中唯一能線性反映 LOD 質量的。

Insight 4 : face_count 達不到目標 ratio

設計取舍。詳見 §6.8.2 解讀。

Insight 5 : Reduce 偶發產生少量法向/拓樸副產物 (TIER 1)

8 inverted + 7 non_manifold = 0.04%/0.02%，極端 collapse 的 fold-over 副作用。視覺不可見、量化誠實。

6.8.5 從直覺到證據的方法論

- **Phase 1 早期**：靠視覺對比 9 組實驗 (A1-A4 / B1-B3 / X1-X2) 找到 ratio + cull + fill 甜蜜點 — 直覺 + 經驗
- **Phase 1 後期**：加 boundary preservation + smart fill — 設計哲學 (治本+補救) 驅動
- **Phase 1 收尾**：實作 diagnose + diff，跑 5Axis baseline — 量化證據驗證

這份 baseline 不是用來修改 preset 的 (視覺判斷已經給出最佳組合)，而是把過去靠肉眼判斷的事用數字佐證：原檔有 22552 islands、115935 boundary edges 是 quantitative root cause；boundary preservation + smart fill 在這個原檔上的對應行為可在 diff 數字裡看到。

未來若有新的工業 CAD FBX：先 diamesh diagnose 看病理 → reduce 三檔 → diamesh diff 對齊 normal_dev 等可信 KPI。

6.8.6 Bridge loops 反直覺實驗 — 量化跟視覺脫鉤

開發過程中加了 `--bridge-loops` 演算法 (用 face strip 連接近距離的 boundary loop , 跟 fill_holes 的「單 loop 三角化」是不同概念) 。在 5Axis 上的對比實驗給了「**量化退、視覺進**」的反直覺結果。

對比指標 (5AxisGlueSpraying.fbx, ratio 0.1, cull 0.025, smart fill) :

指標	不開 bridge	開 bridge	Δ
bridge_pairs_found	–	813	new
bridge_pairs_bridged	–	813 (100%)	✓ 全配對成功
fill_holes_defect_filled	2082	1043	-50%
output_faces	19970	23357	+17%
chamfer	0.080%	0.080%	持平
mean_normal_dev_deg	12.06°	12.81°	+0.75°
視覺輪廓完整度	中	明顯改善	✓
機械臂周圍漂浮	△	✓ 大幅減少	✓

為什麼量化退、視覺進？

bridge 加 face strip 連兩 loop → 新 face 的 normal 跟原檔不對齊 → normal_dev 上升。但這些新 face 補的是「**視覺破口**」，整體輪廓更完整。

這是 §6.6 KPI 紀律的關鍵 case：normal_dev 衡量「像不像原檔」，不是「**視覺完整性**」 — 對 LOD 兩個目標可能矛盾。

bridge 已內建在 tier1/tier2/tier3 preset 中，因為視覺改善壓過量化微退、且 face +17% 對 LOD 場景仍可接受 (30 機產線 30×23k = 690k face，現代 GPU 沒壓力) 。

6.8.7 Negative result — Post-collapse weld 為何沒進 preset

開發過程也試了 `--post-collapse-weld` (在 COLLAPSE 後加一道 weld 修補 close-but-disconnected vertex) 。實驗結果**視覺退化 + 量化退化**，沒進 preset：

指標	不開 post-weld	開 post-weld (multiplier 5.0)
chamfer	0.080%	0.106% (+32%)
mean_normal_dev	12.06°	12.80° (+0.74°)
hausdorff r→o	0.031	0.084 (+170%)
視覺完整度	OK	⚠ 退化 (HMI 旁多異常黑塊)

Root cause：multiplier 5.0 = 0.574 mm 對工業 CAD (CAD seam ~0.1 mm 級) 太大，把不該合的 vertex 也合了，破壞拓樸。

保留 `--post-collapse-weld` flag 為 opt-in，僅在「極端破爛 mesh + multiplier 1.0~1.5」這種 niche 情境值得試。

6.8.8 跟遊戲業 LOD 的差別

遊戲業 LOD 主流做法：

方法	工具 / 引擎	跟 DIAMesh 比
美術手動分檔	Maya / Blender / 3DS Max	高品質、case-by-case，但 30 機 × 多檔 = 不划算
半自動 LOD	Simplygon (業界 SOP，UE/Unity 內建)	比 Blender DECIMATE 智能 — silhouette preservation、UV-aware、material baking
Runtime LOD switching	Unity LODGroup / Unreal LODs	跟 DIAMesh 三檔 preset 對應
Nanite (UE5)	Epic Games	直接放 raw 高模、引擎運行時動態降面 — 行業未來方向

DIAMesh 在哪裡？ * 自動化是我們的 sweet spot — 30 機產線 × 三檔 = 一個批次腳本搞定，遊戲業 30 角色要美術調幾天 * 保材質 + boundary preservation 是我們的設計重點 — Simplygon 也保 UV，但 Simplygon 是專門商業工具 (Epic 收購) * Nanite 路線跟 DIAMesh 互補不衝突 — DIAMesh 給離線 LOD，Nanite 給 runtime；對「不用 UE5 / 內網部署」場景 DIAMesh 仍合適

為什麼不直接用 Simplygon？ * 商業 license + 跟 UE/Unity 綁定 * 對「Python 純自動 + 內網部署」場景不友善 * DIAMesh 用 Blender + trimesh + pymeshlab + 自寫 stage = MIT、跨環境、可內嵌

7. 跨平台部署

7.1 平台支援矩陣

工具	Windows x64	Linux x64	macOS x64	macOS arm64
FBX2glTF	✓ vendored	auto-download	auto-download	auto-download (via x64)
Assimp	✓ vendored	auto-download	auto-download	auto-download
Blender	manual	manual	manual	manual
pyrender	✓	✓	✓	✓
trimesh	✓	✓	✓	✓
pymeshlab	✓ (optional)	✓ (optional)	✓ (optional)	✓ (optional)

7.2 Linux/macOS 一鍵 setup

內網用戶先向開發團隊取得 DIAMesh zip，解壓到本機。

```
cd <unpacked-DIAMesh-directory>
pip install -e .
python scripts/setup_vendor.py    # 自動下載 platform-specific binary (仍需可連 GitHub Release)
# 手動下載 Blender Portable 解壓到 vendor/blender/ (看 vendor/BLENDER_SETUP.md)
diamesh reduce my.fbx --ratio 0.1 --cull-disjoint 0.02 --backend blender -o my_lod.fbx
```

7.3 為什麼不全部 vendor 進 repo

- **Windows .exe / .dll**：~16 MB 總，commit 進 repo (最常見場景，clone 即用)
- **Linux ELF / macOS dylib**：等量大小但每加一個 platform 都要 commit ~10MB+ → 不 sustainable
- **Blender Portable**：~450 MB 解壓後，**且是 GPL** → bundle 進 MIT repo 有 license 感染風險，subprocess 呼叫安全

setup_vendor.py 是 trade-off：repo 保持輕量、Linux/macOS 用戶一行命令補齊。

8. 故障排除

8.1 `ModuleNotFoundError: No module named 'diamesh'`

忘了 `pip install -e .` 。先 `cd` 進 repo 根目錄再 install 。

8.2 `RuntimeError: FBX2glTF binary not found at vendor/fbx2gltf`

在 Linux/macOS 上沒跑 `python scripts/setup_vendor.py` 。

8.3 `pyassimp.errors.AssimpError: assimp library not found`

- Windows : DLL 應該在 `vendor/assimp/` 但 `reducer.py` 沒找到 → 檢查 PATH 或重新 git pull
- Linux/macOS : 跑 `python scripts/setup_vendor.py`

8.4 `RuntimeError: Blender executable not found`

要用 `--backend blender` 但 Blender 沒部署 : * 設環境變數 `BLENDER_EXE=/path/to/blender` * 或下載 Blender Portable 解壓到 `vendor/blender/` * 詳見 `vendor/BLENDER_SETUP.md`

8.5 NumPy 2.x — `AttributeError: np.infty`

DIAMesh 已經 patch 過內嵌 pyrender 。如果您看到這個 error , 可能 pyrender 是 pip-installed 版本 (覆蓋了內嵌版) 。確認 `import pyrender; pyrender.__file__` 指向 `<DIAMesh>/pyrender/__init__.py` 而不是 `site-packages` 。

8.6 reduce 後 mesh 破碎 / 漂浮碎片

- 用 `--backend blender` (trimesh / pymeshlab 不做 mesh repair)
- 加 `--cull-disjoint 0.025` (distance-based island cull)
- 加 `--auto-fill-holes` (補 decimation 留下的 boundary loop)
- 提高 ratio 到 0.25 或 0.5

8.7 reduce 後沒材質

- trimesh / pymeshlab 不保材質 → 換 `--backend blender`
- 輸出選 `.glb` 或 `.fbx` (OBJ/PLY/STL 不帶材質)

8.8 viewer 截圖跟 Win+Shift+S 衝突

DIAMesh 已把 pyrender 截圖鍵從 `S` 改 `P` 。如果您仍碰到衝突 , 看 `pyrender/viewer.py` line 826 確認 patch 已生效 。

9. 限制與未來工作

9.1 目前限制

- 單 mesh 輸出：blender backend 為了跨 part weld 把所有 mesh objects 合併。對「需要 part-by-part 編輯」的 CAD 工作流不友善。LOD 用途無影響。
- 無動畫保留：blender backend `bake_anim=False`，骨架/動畫會丟。想保動畫要改 export 設定。
- Sharp angle 預設 30°：對某些 mesh 過於敏感或不夠敏感 → 寫死，未來可加 `--sharp-angle DEG` flag。
- macOS arm64 + Assimp：setup_vendor.py 抓 `macos-arm64-v6.0.5.zip`，未實機驗證。
- Python 3.14 only：依賴鎖定 Python 3.14。3.11/3.12 應該也可，但沒測。

9.2 路線圖 (ROADMAP.md)

- GUI 整合：viewer 內加 Reduce 按鈕跟 LOD 滑桿，即時預覽
 - Multi-LOD 一次性產出：`diamesh lod machine.fbx --levels 0.5,0.25,0.1` 一次生 3 個 LOD
 - UV-aware simplification：reduce 過程不破壞 UV chart 邊界 (texture-friendly)
 - Voxel remesh fallback：對「mesh topology 太亂」的場景做 voxel reconstruction (重建拓撲)
 - Web viewer：output GLB + Three.js 範例 → 直接 web 可看
-

附錄：完整指令對照

```
# 0. install – 內網用戶向開發團隊取得 zip 後解壓進本機
cd <unpacked-DIAMesh-directory>
pip install -e .

# 1. Linux/macOS only – auto-download platform binaries
python scripts/setup_vendor.py

# 2. Optional – install pymeshlab for the alternate backend
pip install pymeshlab

# 3. Manual – Blender Portable to vendor/blender/ (see vendor/BLENDER_SETUP.md)

# 4. Use
diamesh info <file.fbx>                # quick stats
diamesh view <file.fbx>                 # interactive viewer
diamesh diagnose <file.fbx>             # full health report
                                         # (watertight, non-manifold,
                                         #   inverted_normals, self_intersect,
                                         #   boundary_edges, islands)

# Default backend (trimesh, no material)
diamesh reduce <file.fbx> --target-faces 5000

# 🏆 PRODUCTION – Use the preset, period.
diamesh reduce <file.fbx> --preset tier1   -o <out.fbx>  # 廠區/30 機
diamesh reduce <file.fbx> --preset tier2   -o <out.fbx>  # 單機聚焦
diamesh reduce <file.fbx> --preset balanced -o <out.fbx>  # = tier2 alias
diamesh reduce <file.fbx> --preset tier3   -o <out.fbx>  # Hero shot

# Quantitative deviation between original and LOD
diamesh diff <orig.fbx> <lod.fbx>

# Advanced – opt-in extras (use sparingly, see §6.6)
diamesh reduce <file.fbx> --preset tier1 \
    --fix-non-manifold          `# diagnose 報 non_manifold > 0 時` \
    --fill-holes-smooth         `# 補洞處 Laplacian 平滑（曲面機殼）` \
    -o <out.fbx>
```

```
# Override preset's scalar (e.g. more aggressive face budget)
diamesh reduce <file.fbx> --preset tier1 --ratio 0.05 -o <out.fbx>

# Backend explorations (no material, fastest)
diamesh reduce <file.fbx> --ratio 0.5 --backend pymeshlab -o <out.glb>
diamesh reduce <file.fbx> --ratio 0.5 --backend trimesh -o <out.fbx>
```

版本：2026-05-03 — `--preset` 機制 + bridge loops + 量化-視覺脫鉤紀律 作者：James Chao + Homi (AI Agent)